

State the maximum-flow problem as a linear-programming problem.

### 24.1-6

Professor Adam has two children who, unfortunately, dislike each other. The problem is so severe that not only do they refuse to walk to school together, but in fact each one refuses to walk on any block that the other child has stepped on that day. The children have no problem with their paths crossing at a corner. Fortunately both the professor's house and the school are on corners, but beyond that he is not sure if it is going to be possible to send both of his children to the same school. The professor has a map of his town. Show how to formulate the problem of determining whether both his children can go to the same school as a maximum-flow problem.

### 24.1-7

Suppose that, in addition to edge capacities, a flow network has *vertex capacities*. That is each vertex  $v$  has a limit  $l(v)$  on how much flow can pass through  $v$ . Show how to transform a flow network  $G = (V, E)$  with vertex capacities into an equivalent flow network  $G' = (V', E')$  without vertex capacities, such that a maximum flow in  $G'$  has the same value as a maximum flow in  $G$ . How many vertices and edges does  $G'$  have?

---

## 24.2 The Ford-Fulkerson method

This section presents the Ford-Fulkerson method for solving the maximum-flow problem. We call it a “method” rather than an “algorithm” because it encompasses several implementations with differing running times. The Ford-Fulkerson method depends on three important ideas that transcend the method and are relevant to many flow algorithms and problems: residual networks, augmenting paths, and cuts. These ideas are essential to the important max-flow min-cut theorem (Theorem 24.6), which characterizes the value of a maximum flow in terms of cuts of the flow network. We end this section by presenting one specific implementation of the Ford-Fulkerson method and analyzing its running time.

The Ford-Fulkerson method iteratively increases the value of the flow. It starts with  $f(u, v) = 0$  for all  $u, v \in V$ , giving an initial flow of value 0. Each iteration increases the flow value in  $G$  by finding an “augmenting path” in an associated “residual network”  $G_f$ . The edges of the augmenting path in  $G_f$  indicate on which edges in  $G$  to update the flow in order to increase the flow value. Although each iteration of the Ford-Fulkerson method increases the value of the flow, we’ll see that the flow on any particular edge of  $G$  may increase or decrease. Although it might seem counterintuitive to decrease the flow on an edge, doing so may enable flow to increase on other edges, allowing more flow to travel from the source to the sink. The Ford-Fulkerson method, given in the procedure FORD-FULKERSON-METHOD, repeatedly augments the flow until the residual network has no more augmenting paths. The max-flow min-cut theorem shows that upon termination, this process yields a maximum flow.

#### FORD-FULKERSON-METHOD ( $G, s, t$ )

```
1 initialize flow  $f$  to 0
2 while there exists an augmenting path  $p$  in the residual network  $G_f$ 
3     augment flow  $f$  along  $p$ 
4 return  $f$ 
```

In order to implement and analyze the Ford-Fulkerson method, we need to introduce several additional concepts.

### Residual networks

Intuitively, given a flow network  $G$  and a flow  $f$ , the residual network  $G_f$  consists of edges whose capacities represent how the flow can change on edges of  $G$ . An edge of the flow network can admit an amount of additional flow equal to the edge’s capacity minus the flow on that edge. If that value is positive, that edge goes into  $G_f$  with a “residual capacity” of  $c_f(u, v) = c(u, v) - f(u, v)$ . The only edges of  $G$  that belong

to  $G_f$  are those that can admit more flow. Those edges  $(u, v)$  whose flow equals their capacity have  $c_f(u, v) = 0$ , and they do not belong to  $G_f$ .

You might be surprised that the residual network  $G_f$  can also contain edges that are not in  $G$ . As an algorithm manipulates the flow, with the goal of increasing the total flow, it might need to decrease the flow on a particular edge in order to increase the flow elsewhere. In order to represent a possible decrease in the positive flow  $f(u, v)$  on an edge in  $G$ , the residual network  $G_f$  contains an edge  $(v, u)$  with residual capacity  $c_f(v, u) = f(u, v)$ —that is, an edge that can admit flow in the opposite direction to  $(u, v)$ , at most canceling out the flow on  $(u, v)$ . These reverse edges in the residual network allow an algorithm to send back flow it has already sent along an edge. Sending flow back along an edge is equivalent to *decreasing* the flow on the edge, which is a necessary operation in many algorithms.

More formally, for a flow network  $G = (V, E)$  with source  $s$ , sink  $t$ , and a flow  $f$ , consider a pair of vertices  $u, v \in V$ . We define the **residual capacity**  $c_f(u, v)$  by

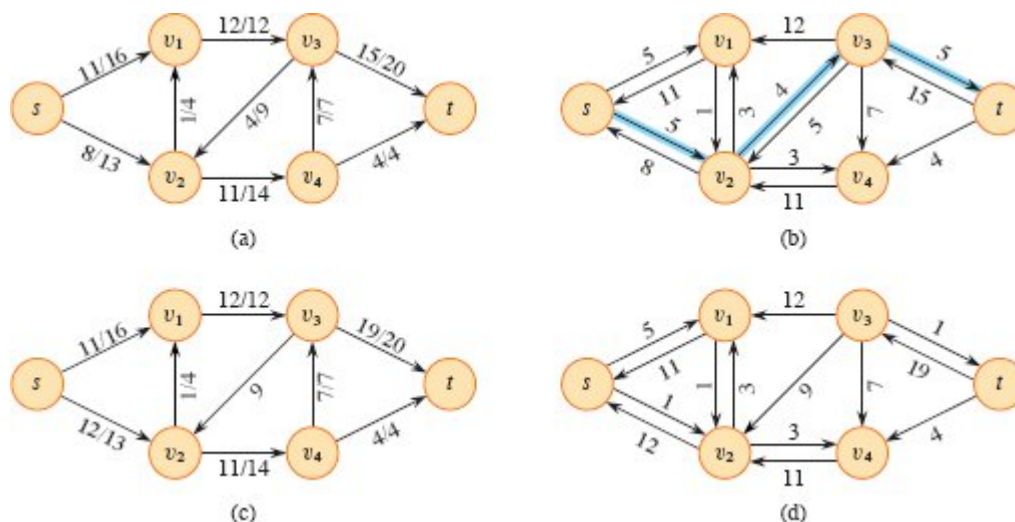
$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise.} \end{cases} \quad (24.2)$$

In a flow network,  $(u, v) \in E$  implies  $(v, u) \notin E$ , and so exactly one case in equation (24.2) applies to each ordered pair of vertices.

As an example of equation (24.2), if  $c(u, v) = 16$  and  $f(u, v) = 11$ , then  $f(u, v)$  can increase by up to  $c_f(u, v) = 5$  units before exceeding the capacity constraint on edge  $(u, v)$ . Alternatively, up to 11 units of flow can return from  $v$  to  $u$ , so that  $c_f(v, u) = 11$ .

Given a flow network  $G = (V, E)$  and a flow  $f$ , the **residual network** of  $G$  induced by  $f$  is  $G_f = (V, E_f)$ , where

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}. \quad (24.3)$$



**Figure 24.4** (a) The flow network  $G$  and flow  $f$  of Figure 24.1(b). (b) The residual network  $G_f$  with augmenting path  $p$ , having residual capacity  $c_f(p) = c_f(v_2, v_3) = 4$ , in blue. Edges with residual capacity equal to 0, such as  $(v_1, v_3)$ , are not shown, a convention we follow in the remainder of this section. (c) The flow in  $G$  that results from augmenting along path  $p$  by its residual capacity 4. Edges carrying no flow, such as  $(v_3, v_2)$ , are labeled only by their capacity, another convention we follow throughout. (d) The residual network induced by the flow in (c).

That is, as promised above, each edge of the residual network, or **residual edge**, can admit a flow that is greater than 0. Figure 24.4(a) repeats the flow network  $G$  and flow  $f$  of Figure 24.1(b), and Figure 24.4(b) shows the corresponding residual network  $G_f$ . The edges in  $E_f$  are either edges in  $E$  or their reversals, and thus

$$|E_f| \leq 2 |E|.$$

Observe that the residual network  $G_f$  is similar to a flow network with capacities given by  $c_f$ . It does not satisfy the definition of a flow network, however, because it could contain antiparallel edges. Other than this difference, a residual network has the same properties as a flow network, and we can define a flow in the residual network as one that satisfies the definition of a flow, but with respect to capacities  $c_f$  in the residual network  $G_f$ .

A flow in a residual network provides a roadmap for adding flow to the original flow network. If  $f$  is a flow in  $G$  and  $f'$  is a flow in the

corresponding residual network  $G_f$ , we define  $f \uparrow f'$ , the *augmentation* of flow  $f$  by  $f'$ , to be a function from  $V \times V$  to  $\mathbb{R}$ , defined by

$$(f \uparrow f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u) & \text{if } (u, v) \in E, \\ 0 & \text{otherwise.} \end{cases} \quad (24.4)$$

The intuition behind this definition follows the definition of the residual network. The flow on  $(u, v)$  increases by  $f'(u, v)$ , but decreases by  $f'(v, u)$  because pushing flow on the reverse edge in the residual network signifies decreasing the flow in the original network. Pushing flow on the reverse edge in the residual network is also known as *cancellation*. For example, suppose that 5 crates of hockey pucks go from  $u$  to  $v$  and 2 crates go from  $v$  to  $u$ . That is equivalent (from the perspective of the final result) to sending 3 crates from  $u$  to  $v$  and none from  $v$  to  $u$ . Cancellation of this type is crucial for any maximum-flow algorithm.

The following lemma shows that augmenting a flow in  $G$  by a flow in  $G_f$  yields a new flow in  $G$  with a greater flow value.

**Lemma 24.1**

Let  $G = (V, E)$  be a flow network with source  $s$  and sink  $t$ , and let  $f$  be a flow in  $G$ . Let  $G_f$  be the residual network of  $G$  induced by  $f$ , and let  $f'$  be a flow in  $G_f$ . Then the function  $f \uparrow f'$  defined in equation (24.4) is a flow in  $G$  with value  $|f \uparrow f'| = |f| + |f'|$ .

**Proof** We first verify that  $f \uparrow f'$  obeys the capacity constraint for each edge in  $E$  and flow conservation at each vertex in  $V - \{s, t\}$ .

For the capacity constraint, first observe that if  $(u, v) \in E$ , then  $c_f(v, u) = f(u, v)$ . Because  $f'$  is a flow in  $G_f$ , we have  $f'(v, u) \leq c_f(v, u)$ , which gives  $f'(v, u) \leq f(u, v)$ . Therefore,

$$\begin{aligned} (f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \text{ (by equation (24.4))} \\ &\geq f(u, v) + f'(u, v) - f(u, v) \text{ (because } f'(v, u) \leq f(u, v)) \\ &= f'(u, v) \\ &\geq 0. \end{aligned}$$

In addition,

$$\begin{aligned}
(f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \quad (\text{by equation (24.4)}) \\
&\leq f(u, v) + f'(u, v) \quad (\text{because flows are nonnegative}) \\
&\leq f(u, v) + c_f(u, v) \quad (\text{capacity constraint}) \\
&= f(u, v) + c(u, v) - f(u, v) \quad (\text{definition of } c_f) \\
&= c(u, v).
\end{aligned}$$

To show that flow conservation holds and that  $|f \uparrow f'| = |f| + |f'|$ , we first prove the claim that for all  $u \in V$ , we have

$$\begin{aligned}
\sum_{v \in V} (f \uparrow f')(u, v) - \sum_{v \in V} (f \uparrow f')(v, u) \\
= \sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) + \sum_{v \in V} f'(u, v) - \sum_{v \in V} f'(v, u). \quad (24.5)
\end{aligned}$$

Because we disallow antiparallel edges in  $G$  (but not in  $G_f$ ), we know that for each vertex  $u$ , there can be an edge  $(u, v)$  or  $(v, u)$  in  $G$ , but never both. For a fixed vertex  $u$ , define  $V_l(u) = \{v : (u, v) \in E\}$  to be the set of vertices with edges in  $G$  leaving  $u$ , and define  $V_e(u) = \{v : (v, u) \in E\}$  to be the set of vertices with edges in  $G$  entering  $u$ . We have  $V_l(u) \cup V_e(u) \subseteq V$  and, because  $G$  contains no antiparallel edges,  $V_l(u) \cap V_e(u) = \emptyset$ . By the definition of flow augmentation in equation (24.4), only vertices  $v$  in  $V_l(u)$  can have positive  $(f \uparrow f')(u, v)$ , and only vertices  $v$  in  $V_e(u)$  can have positive  $(f \uparrow f')(v, u)$ . Starting from the left-hand side of equation (24.5), we use this fact and then reorder and group terms, giving

$$\begin{aligned}
& \sum_{v \in V} (f \uparrow f')(u, v) - \sum_{v \in V} (f \uparrow f')(v, u) \\
&= \sum_{v \in V_I(u)} (f \uparrow f')(u, v) - \sum_{v \in V_E(u)} (f \uparrow f')(v, u) \\
&= \sum_{v \in V_I(u)} (f(u, v) + f'(u, v) - f'(v, u)) - \sum_{v \in V_E(u)} (f(v, u) + f'(v, u) - f'(u, v)) \\
&= \sum_{v \in V_I(u)} f(u, v) + \sum_{v \in V_I(u)} f'(u, v) - \sum_{v \in V_I(u)} f'(v, u) \\
&\quad - \sum_{v \in V_E(u)} f(v, u) - \sum_{v \in V_E(u)} f'(v, u) + \sum_{v \in V_E(u)} f'(u, v) \\
&= \sum_{v \in V_I(u)} f(u, v) - \sum_{v \in V_E(u)} f(v, u) \\
&\quad + \sum_{v \in V_I(u)} f'(u, v) + \sum_{v \in V_E(u)} f'(u, v) - \sum_{v \in V_I(u)} f'(v, u) - \sum_{v \in V_E(u)} f'(v, u) \\
&= \sum_{v \in V_I(u)} f(u, v) - \sum_{v \in V_E(u)} f(v, u) + \sum_{v \in V_I(u) \cup V_E(u)} f'(u, v) - \sum_{v \in V_I(u) \cup V_E(u)} f'(v, u) . \tag{24.6}
\end{aligned}$$

In equation (24.6), all four summations can extend to sum over  $V$ , since each additional term has value 0. (Exercise 24.2-1 asks you to prove this formally.) Taking all four summations over  $V$ , instead of just subsets of  $V$ , proves the claim in equation (24.5).

Now we are ready to prove flow conservation for  $f \uparrow f'$  and that  $|f \uparrow f'| = |f| + |f'|$ . For the latter property, let  $u = s$  in equation (24.5). Then, we have

$$\begin{aligned}
|f \uparrow f'| &= \sum_{v \in V} (f \uparrow f')(s, v) - \sum_{v \in V} (f \uparrow f')(v, s) \\
&= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{v \in V} f'(s, v) - \sum_{v \in V} f'(v, s) \\
&= |f| + |f'| .
\end{aligned}$$

For flow conservation, observe that for any vertex  $u$  that is neither  $s$  nor  $t$ , flow conservation for  $f$  and  $f'$  means that the right-hand side of equation (24.5) is 0, and thus  $\sum_{v \in V} (f \uparrow f')(u, v) = \sum_{v \in V} (f \uparrow f')(v, u)$ . ■

## Augmenting paths

Given a flow network  $G = (V, E)$  and a flow  $f$ , an **augmenting path**  $p$  is a simple path from  $s$  to  $t$  in the residual network  $G_f$ . By the definition of the residual network, the flow on an edge  $(u, v)$  of an augmenting path

may increase by up to  $c_f(u, v)$  without violating the capacity constraint on whichever of  $(u, v)$  and  $(v, u)$  belongs to the original flow network  $G$ .

The blue path in Figure 24.4(b) is an augmenting path. Treating the residual network  $G_f$  in the figure as a flow network, the flow through each edge of this path can increase by up to 4 units without violating a capacity constraint, since the smallest residual capacity on this path is  $c_f(v_2, v_3) = 4$ . We call the maximum amount by which we can increase the flow on each edge in an augmenting path  $p$  the *residual capacity* of  $p$ , given by

$$c_f(p) = \min \{c_f(u, v) : (u, v) \text{ is in } p\}.$$

The following lemma, which Exercise 24.2-7 asks you to prove, makes the above argument more precise.

**Lemma 24.2**

Let  $G = (V, E)$  be a flow network, let  $f$  be a flow in  $G$ , and let  $p$  be an augmenting path in  $G_f$ . Define a function  $f_p : V \times V \rightarrow \mathbb{R}$  by

$$f_p(u, v) = \begin{cases} c_f(p) & \text{if } (u, v) \text{ is on } p, \\ 0 & \text{otherwise.} \end{cases} \quad (24.7)$$

Then,  $f_p$  is a flow in  $G_f$  with value  $|f_p| = c_f(p) > 0$ .

■

The following corollary shows that augmenting  $f$  by  $f_p$  produces another flow in  $G$  whose value is closer to the maximum. Figure 24.4(c) shows the result of augmenting the flow  $f$  from Figure 24.4(a) by the flow  $f_p$  in Figure 24.4(b), and Figure 24.4(d) shows the ensuing residual network.

**Corollary 24.3**

Let  $G = (V, E)$  be a flow network, let  $f$  be a flow in  $G$ , and let  $p$  be an augmenting path in  $G_f$ . Let  $f_p$  be defined as in equation (24.7), and suppose that  $f$  is augmented by  $f_p$ . Then the function  $f \uparrow f_p$  is a flow in  $G$  with value  $|f \uparrow f_p| = |f| + |f_p| > |f|$ .



**Proof** Immediate from Lemmas 24.1 and 24.2. ■

## Cuts of flow networks

The Ford-Fulkerson method repeatedly augments the flow along augmenting paths until it has found a maximum flow. How do we know that when the algorithm terminates, it has actually found a maximum flow? The max-flow min-cut theorem, which we will prove shortly, tells us that a flow is maximum if and only if its residual network contains no augmenting path. To prove this theorem, though, we must first explore the notion of a cut of a flow network.

A **cut**  $(S, T)$  of flow network  $G = (V, E)$  is a partition of  $V$  into  $S$  and  $T = V - S$  such that  $s \in S$  and  $t \in T$ . (This definition is similar to the definition of “cut” that we used for minimum spanning trees in Chapter 21, except that here we are cutting a directed graph rather than an undirected graph, and we insist that  $s \in S$  and  $t \in T$ .) If  $f$  is a flow, then the **net flow**  $f(S, T)$  across the cut  $(S, T)$  is defined to be

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u). \quad (24.8)$$

The **capacity** of the cut  $(S, T)$  is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v). \quad (24.9)$$

A **minimum cut** of a network is a cut whose capacity is minimum over all cuts of the network.

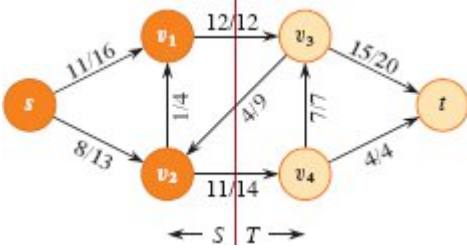
You probably noticed that the definitions of flow across a cut and capacity of a cut differ in that flow counts edges going in both directions across the cut, but capacity counts only edges going from the source side of the cut toward the sink side. This asymmetry is intentional and important. The reason for this difference will become apparent later in this section.

Figure 24.5 shows the cut  $(\{s, v_1, v_2\}, \{v_3, v_4, t\})$  in the flow network of Figure 24.1(b). The net flow across this cut is

$$f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) = 12 + 11 - 4$$

$$= 19,$$

and the capacity of this cut is



**Figure 24.5** A cut  $(S, T)$  in the flow network of Figure 24.1(b), where  $S = \{s, v_1, v_2\}$  and  $T = \{v_3, v_4, t\}$ . The vertices in  $S$  are orange, and the vertices in  $T$  are tan. The net flow across  $(S, T)$  is  $f(S, T) = 19$ , and the capacity is  $c(S, T) = 26$ .

$$\begin{aligned} c(v_1, v_3) + c(v_2, v_4) &= 12 + 14 \\ &= 26. \end{aligned}$$

The following lemma shows that, for a given flow  $f$ , the net flow across any cut is the same, and it equals  $|f|$ , the value of the flow.

**Lemma 24.4**

Let  $f$  be a flow in a flow network  $G$  with source  $s$  and sink  $t$ , and let  $(S, T)$  be any cut of  $G$ . Then the net flow across  $(S, T)$  is  $f(S, T) = |f|$ .

**Proof** For any vertex  $u \in V - \{s, t\}$ , rewrite the flow-conservation condition as

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0. \quad (24.10)$$

Taking the definition of  $|f|$  from equation (24.1) and adding the left-hand side of equation (24.10), which equals 0, summed over all vertices in  $S - \{s\}$ , gives

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left( \sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right).$$

Expanding the right-hand summation and regrouping terms yields

$$\begin{aligned}
|f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\
&= \sum_{v \in V} \left( f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left( f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\
&= \sum_{v \in V} \sum_{u \in S} f(u, v) - \sum_{v \in V} \sum_{u \in S} f(v, u) .
\end{aligned}$$

Because  $V = S \cup T$  and  $S \cap T = \emptyset$ , splitting each summation over  $V$  into summations over  $S$  and  $T$  gives

$$\begin{aligned}
|f| &= \sum_{v \in S} \sum_{u \in S} f(u, v) + \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\
&= \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\
&\quad + \left( \sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) \right) .
\end{aligned}$$

The two summations within the parentheses are actually the same, since for all vertices  $x, y \in S$ , the term  $f(x, y)$  appears once in each summation. Hence, these summations cancel, yielding

$$\begin{aligned}
|f| &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\
&= f(S, T) .
\end{aligned}$$

■

A corollary to Lemma 24.4 shows how cut capacities bound the value of a flow.

### **Corollary 24.5**

The value of any flow  $f$  in a flow network  $G$  is bounded from above by the capacity of any cut of  $G$ .

**Proof** Let  $(S, T)$  be any cut of  $G$  and let  $f$  be any flow. By Lemma 24.4 and the capacity constraint,

$$\begin{aligned}
|f| &= f(S, T) \\
&= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\
&\leq \sum_{u \in S} \sum_{v \in T} f(u, v) \\
&\leq \sum_{u \in S} \sum_{v \in T} c(u, v) \\
&= c(S, T).
\end{aligned}$$

■

Corollary 24.5 yields the immediate consequence that the value of a maximum flow in a network is bounded from above by the capacity of a minimum cut of the network. The important max-flow min-cut theorem, which we now state and prove, says that the value of a maximum flow is in fact equal to the capacity of a minimum cut.

***Theorem 24.6 (Max-flow min-cut theorem)***

If  $f$  is a flow in a flow network  $G = (V, E)$  with source  $s$  and sink  $t$ , then the following conditions are equivalent:

1.  $f$  is a maximum flow in  $G$ .
2. The residual network  $G_f$  contains no augmenting paths.
3.  $|f| = c(S, T)$  for some cut  $(S, T)$  of  $G$ .

**Proof** (1)  $\Rightarrow$  (2): Suppose for the sake of contradiction that  $f$  is a maximum flow in  $G$  but that  $G_f$  has an augmenting path  $p$ . Then, by Corollary 24.3, the flow found by augmenting  $f$  by  $f_p$ , where  $f_p$  is given by equation (24.7), is a flow in  $G$  with value strictly greater than  $|f|$ , contradicting the assumption that  $f$  is a maximum flow.

(2)  $\Rightarrow$  (3): Suppose that  $G_f$  has no augmenting path, that is, that  $G_f$  contains no path from  $s$  to  $t$ . Define

$$S = \{v \in V : \text{there exists a path from } s \text{ to } v \text{ in } G_f\}$$

and  $T = V - S$ . The partition  $(S, T)$  is a cut: we have  $s \in S$  trivially and  $t \notin S$  because there is no path from  $s$  to  $t$  in  $G_f$ . Now consider a pair of vertices  $u \in S$  and  $v \in T$ . If  $(u, v) \in E$ , we must have  $f(u, v) = c(u, v)$ ,

since otherwise  $(u, v) \in E_f$ , which would place  $v$  in set  $S$ . If  $(v, u) \in E$ , we must have  $f(v, u) = 0$ , because otherwise  $c_f(u, v) = f(v, u)$  would be positive and we would have  $(u, v) \in E_f$ , which again would place  $v$  in  $S$ . Of course, if neither  $(u, v)$  nor  $(v, u)$  belongs to  $E$ , then  $f(u, v) = f(v, u) = 0$ . We thus have

$$\begin{aligned} f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 \\ &= c(S, T). \end{aligned}$$

By Lemma 24.4, therefore,  $|f| = f(S, T) = c(S, T)$ .

(3)  $\Rightarrow$  (1): By Corollary 24.5,  $|f| \leq c(S, T)$  for all cuts  $(S, T)$ . The condition  $|f| = c(S, T)$  thus implies that  $f$  is a maximum flow. ■

### The basic Ford-Fulkerson algorithm

Each iteration of the Ford-Fulkerson method finds *some* augmenting path  $p$  and uses  $p$  to modify the flow  $f$ . As Lemma 24.2 and Corollary 24.3 suggest, replacing  $f$  by  $f \uparrow f_p$  produces a new flow whose value is  $|f| + |f_p|$ . The procedure FORD-FULKERSON on the next page implements the method by updating the flow attribute  $(u, v).f$  for each edge  $(u, v) \in E$ .<sup>1</sup> It assumes implicitly that  $(u, v).f = 0$  if  $(u, v) \notin E$ . The procedure also assumes that the capacities  $c(u, v)$  come with the flow network, and that  $c(u, v) = 0$  if  $(u, v) \notin E$ . The procedure computes the residual capacity  $c_f(u, v)$  in accordance with the formula (24.2). The expression  $c_f(p)$  in the code is just a temporary variable that stores the residual capacity of the path  $p$ .

**FORD-FULKERSON** ( $G, s, t$ )

```

1 for each edge  $(u, v) \in G.E$ 
2    $(u, v).f = 0$ 
3 while there exists a path  $p$  from  $s$  to  $t$  in the residual network  $G_f$ 
4    $c_f(p) = \min \{c_f(u, v) : (u, v) \text{ is in } p\}$ 
```

```

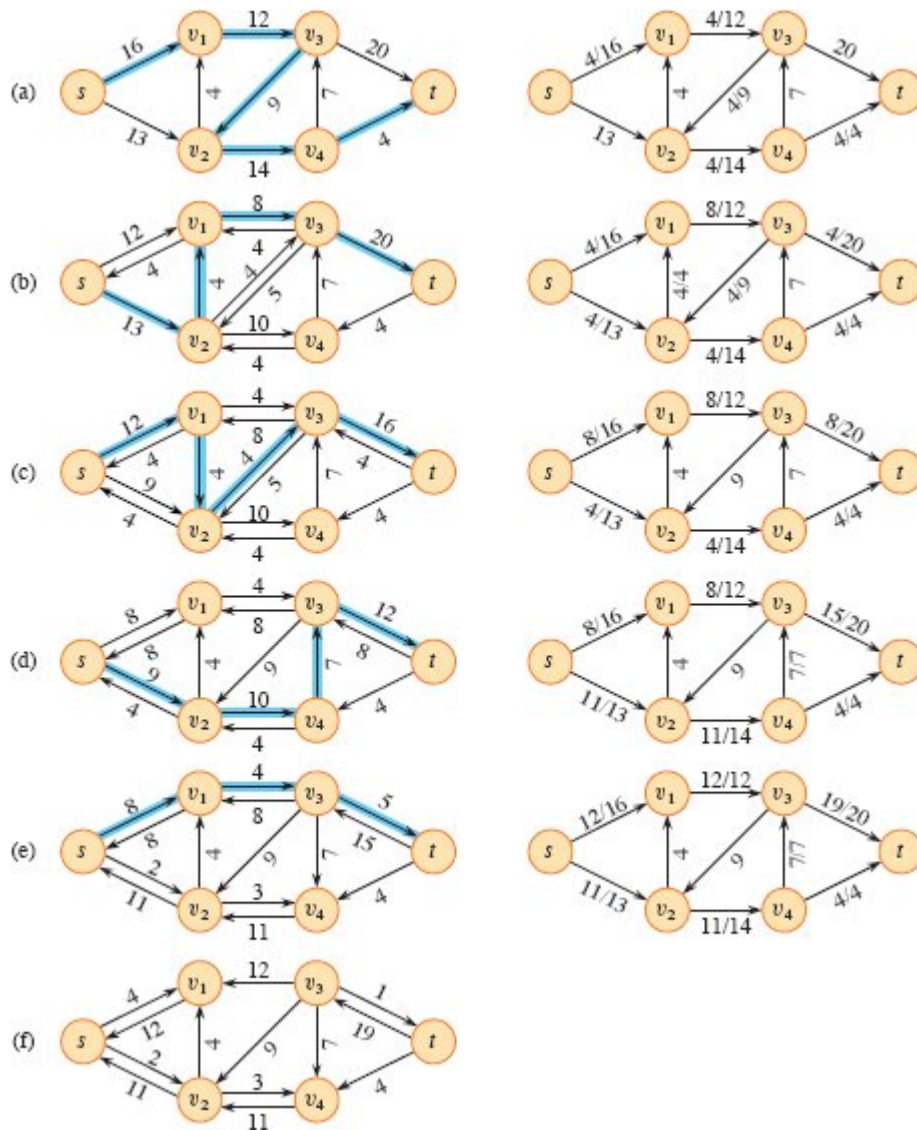
5   for each edge  $(u, v)$  in  $p$ 
6       if  $(u, v) \in G.E$ 
7            $(u, v).f = (u, v).f + c_f(p)$ 
8       else  $(v, u).f = (v, u).f - c_f(p)$ 
9   return  $f$ 

```

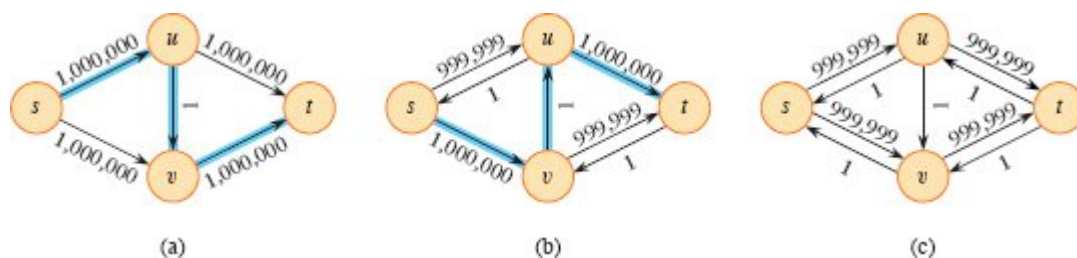
The FORD-FULKERSON procedure simply expands on the FORD-FULKERSON-METHOD pseudocode given earlier. Figure 24.6 shows the result of each iteration in a sample run. Lines 1–2 initialize the flow  $f$  to 0. The **while** loop of lines 3–8 repeatedly finds an augmenting path  $p$  in  $G_f$  and augments flow  $f$  along  $p$  by the residual capacity  $c_f(p)$ . Each residual edge in path  $p$  is either an edge in the original network or the reversal of an edge in the original network. Lines 6–8 update the flow in each case appropriately, adding flow when the residual edge is an original edge and subtracting it otherwise. When no augmenting paths exist, the flow  $f$  is a maximum flow.

### Analysis of Ford-Fulkerson

The running time of FORD-FULKERSON depends on the augmenting path  $p$  and how it's found in line 3. If the edge capacities are irrational numbers, it's possible to choose the augmenting path so that the algorithm never terminates: the value of the flow increases with successive augmentations, but never converges to the maximum flow value. The good news is that if the algorithm finds the augmenting path by using a breadth-first search (which we saw in Section 20.2), it runs in polynomial time. Before proving this result, we obtain a simple bound for the case in which all capacities are integers and the algorithm finds any augmenting path.



**Figure 24.6** The execution of the basic Ford-Fulkerson algorithm. **(a)–(e)** Successive iterations of the **while** loop. The left side of each part shows the residual network  $G_f$  from line 3 with a blue augmenting path  $p$ . The right side of each part shows the new flow  $f$  that results from augmenting  $f$  by  $f_p$ . The residual network in (a) is the input flow network  $G$ . **(f)** The residual network at the last **while** loop test. It has no augmenting paths, and the flow  $f$  shown in (e) is therefore a maximum flow. The value of the maximum flow found is 23.



**Figure 24.7** (a) A flow network for which FORD-FULKERSON can take  $\Theta(E |f^*|)$  time, where  $f^*$  is a maximum flow, shown here with  $|f^*| = 2,000,000$ . The blue path is an augmenting path with residual capacity 1. (b) The resulting residual network, with another augmenting path whose residual capacity is 1. (c) The resulting residual network.

In practice, the maximum-flow problem often arises with integer capacities. If the capacities are rational numbers, an appropriate scaling transformation can make them all integers. If  $f^*$  denotes a maximum flow in the transformed network, then a straightforward implementation of FORD-FULKERSON executes the **while** loop of lines 3–8 at most  $|f^*|$  times, since the flow value increases by at least 1 unit in each iteration.

A good implementation should perform the work done within the **while** loop efficiently. It should represent the flow network  $G = (V, E)$  with the right data structure and find an augmenting path by a linear-time algorithm. Let's assume that the implementation keeps a data structure corresponding to a directed graph  $G' = (V, E')$ , where  $E' = \{(u, v) : (u, v) \in E \text{ or } (v, u) \in E\}$ . Edges in the network  $G$  are also edges in  $G'$ , making it straightforward to maintain capacities and flows in this data structure. Given a flow  $f$  on  $G$ , the edges in the residual network  $G_f$  consist of all edges  $(u, v)$  of  $G'$  such that  $c_f(u, v) > 0$ , where  $c_f$  conforms to equation (24.2). The time to find a path in a residual network is therefore  $O(V + E') = O(E)$  using either depth-first search or breadth-first search. Each iteration of the **while** loop thus takes  $O(E)$  time, as does the initialization in lines 1–2, making the total running time of the FORD-FULKERSON algorithm  $O(E |f^*|)$ .

When the capacities are integers and the optimal flow value  $|f^*|$  is small, the running time of the Ford-Fulkerson algorithm is good. Figure 24.7(a) shows an example of what can happen on a simple flow network



for which  $|f^*|$  is large. A maximum flow in this network has value 2,000,000: 1,000,000 units of flow traverse the path  $s \rightarrow u \rightarrow t$ , and another 1,000,000 units traverse the path  $s \rightarrow v \rightarrow t$ . If the first augmenting path found by FORD-FULKERSON is  $s \rightarrow u \rightarrow v \rightarrow t$ , shown in Figure 24.7(a), the flow has value 1 after the first iteration. The resulting residual network appears in Figure 24.7(b). If the second iteration finds the augmenting path  $s \rightarrow v \rightarrow u \rightarrow t$ , as shown in Figure 24.7(b), the flow then has value 2. Figure 24.7(c) shows the resulting residual network. If the algorithm continues alternately choosing the augmenting paths  $s \rightarrow u \rightarrow v \rightarrow t$  and  $s \rightarrow v \rightarrow u \rightarrow t$ , it performs a total of 2,000,000 augmentations, increasing the flow value by only 1 unit in each.

### The Edmonds-Karp algorithm

In the example of Figure 24.7, the algorithm never chooses the augmenting path with the fewest edges. It should have. By using breadth-first search to find an augmenting path in the residual network, the algorithm runs in polynomial time, independent of the maximum flow value. We call the Ford-Fulkerson method so implemented the *Edmonds-Karp algorithm*.

Let's now prove that the Edmonds-Karp algorithm runs in  $O(VE^2)$  time. The analysis depends on the distances to vertices in the residual network  $G_f$ . The notation  $\delta_f(u, v)$  denotes the shortest-path distance from  $u$  to  $v$  in  $G_f$ , where each edge has unit distance.

#### **Lemma 24.7**

If the Edmonds-Karp algorithm is run on a flow network  $G = (V, E)$  with source  $s$  and sink  $t$ , then for all vertices  $v \in V - \{s, t\}$ , the shortest-path distance  $\delta_f(s, v)$  in the residual network  $G_f$  increases monotonically with each flow augmentation.

**Proof** We'll suppose that a flow augmentation occurs that causes the shortest-path distance from  $s$  to some vertex  $v \in V - \{s, t\}$  to decrease and then derive a contradiction. Let  $f$  be the flow just before an augmentation that decreases some shortest-path distance, and let  $f'$  be

the flow just afterward. Let  $v$  be a vertex with the minimum  $\delta_{f'}(s, v)$  whose distance was decreased by the augmentation, so that  $\delta_{f'}(s, v) < \delta_f(s, v)$ . Let  $p = s \rightsquigarrow u \rightarrow v$  be a shortest path from  $s$  to  $v$  in  $G_{f'}$ , so that  $(u, v) \in E_{f'}$  and

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1. \quad (24.11)$$

Because of how we chose  $v$ , we know that the distance of vertex  $u$  from the source  $s$  did not decrease, that is,

$$\delta_{f'}(s, u) \geq \delta_f(s, u). \quad (24.12)$$

We claim that  $(u, v) \notin E_f$ . Why? If we have  $(u, v) \in E_f$ , then we also have

$$\begin{aligned} \delta_f(s, v) &\leq \delta_f(s, u) + 1 \quad (\text{by Lemma 22.10, the triangle inequality}) \\ &\leq \delta_{f'}(s, u) + 1 \quad (\text{by inequality (24.12)}) \\ &= \delta_{f'}(s, v) \quad (\text{by equation (24.11)}), \end{aligned}$$

which contradicts our assumption that  $\delta_{f'}(s, v) < \delta_f(s, v)$ .

How can we have  $(u, v) \notin E_f$  and  $(u, v) \in E_{f'}$ ? The augmentation must have increased the flow from  $v$  to  $u$ , so that edge  $(v, u)$  was in the augmenting path. The augmenting path was a shortest path from  $s$  to  $t$  in  $G_f$ , and since any subpath of a shortest path is itself a shortest path, this augmenting path includes a shortest path from  $s$  to  $u$  in  $G_f$  that has  $(v, u)$  as its last edge. Therefore,

$$\begin{aligned} \delta_f(s, v) &= \delta_f(s, u) - 1 \\ &\leq \delta_{f'}(s, u) - 1 \quad (\text{by inequality (24.12)}) \\ &= \delta_{f'}(s, v) - 2 \quad (\text{by equation (24.11)}), \end{aligned}$$

so that  $\delta_{f'}(s, v) > \delta_f(s, v)$ , contradicting our assumption that  $\delta_{f'}(s, v) < \delta_f(s, v)$ . We conclude that our assumption that such a vertex  $v$  exists is incorrect. ■

The next theorem bounds the number of iterations of the Edmonds-Karp algorithm.

**Theorem 24.8**

If the Edmonds-Karp algorithm is run on a flow network  $G = (V, E)$  with source  $s$  and sink  $t$ , then the total number of flow augmentations performed by the algorithm is  $O(VE)$ .

**Proof** We say that an edge  $(u, v)$  in a residual network  $G_f$  is *critical* on an augmenting path  $p$  if the residual capacity of  $p$  is the residual capacity of  $(u, v)$ , that is, if  $c_f(p) = c_f(u, v)$ . After flow is augmented along an augmenting path, any critical edge on the path disappears from the residual network. Moreover, at least one edge on any augmenting path must be critical. We'll show that each of the  $|E|$  edges can become critical at most  $|V|/2$  times.

Let  $u$  and  $v$  be vertices in  $V$  that are connected by an edge in  $E$ . Since augmenting paths are shortest paths, when  $(u, v)$  is critical for the first time, we have

$$\delta_f(s, v) = \delta_f(s, u) + 1.$$

Once the flow is augmented, the edge  $(u, v)$  disappears from the residual network. It cannot reappear later on another augmenting path until after the flow from  $u$  to  $v$  is decreased, which occurs only if  $(v, u)$  appears on an augmenting path. If  $f'$  is the flow in  $G$  when this event occurs, then we have

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1.$$

Since  $\delta_f(s, v) \leq \delta_{f'}(s, v)$  by Lemma 24.7, we have

$$\begin{aligned} \delta_{f'}(s, u) &= \delta_{f'}(s, v) + 1 \\ &\geq \delta_f(s, v) + 1 \\ &= \delta_f(s, u) + 2. \end{aligned}$$

Consequently, from the time  $(u, v)$  becomes critical to the time when it next becomes critical, the distance of  $u$  from the source increases by

at least 2. The distance of  $u$  from the source is initially at least 0. Because edge  $(u, v)$  is on an augmenting path, and augmenting paths end at  $t$ , we know that  $u$  cannot be  $t$ , so that in any residual network that has a path from  $s$  to  $u$ , the shortest such path has at most  $|V| - 2$  edges. Thus, after the first time that  $(u, v)$  becomes critical, it can become critical at most  $(|V| - 2)/2 = |V|/2 - 1$  times more, for a total of at most  $|V|/2$  times. Since there are  $O(E)$  pairs of vertices that can have an edge between them in a residual network, the total number of critical edges during the entire execution of the Edmonds-Karp algorithm is  $O(VE)$ . Each augmenting path has at least one critical edge, and hence the theorem follows. ■

Because each iteration of FORD-FULKERSON takes  $O(E)$  time when it uses breadth-first search to find the augmenting path, the total running time of the Edmonds-Karp algorithm is  $O(VE^2)$ .

## Exercises

### 24.2-1

Prove that the summations in equation (24.6) equal the summations on the right-hand side of equation (24.5).

### 24.2-2

In Figure 24.1(b), what is the net flow across the cut  $(\{s, v_2, v_4\}, \{v_1, v_3, t\})$ ? What is the capacity of this cut?

### 24.2-3

Show the execution of the Edmonds-Karp algorithm on the flow network of Figure 24.1(a).

### 24.2-4

In the example of Figure 24.6, what is the minimum cut corresponding to the maximum flow shown? Of the augmenting paths appearing in the example, which one cancels flow?

### 24.2-5

The construction in Section 24.1 to convert a flow network with multiple sources and sinks into a single-source, single-sink network adds edges with infinite capacity. Prove that any flow in the resulting network has a finite value if the edges of the original network with multiple sources and sinks have finite capacity.

#### 24.2-6

Suppose that each source  $s_i$  in a flow network with multiple sources and sinks produces exactly  $p_i$  units of flow, so that  $\sum_{v \in V} f(s_i, v) = p_i$ . Suppose also that each sink  $t_j$  consumes exactly  $q_j$  units, so that  $\sum_{v \in V} f(v, t_j) = q_j$ , where  $\sum_i p_i = \sum_j q_j$ . Show how to convert the problem of finding a flow  $f$  that obeys these additional constraints into the problem of finding a maximum flow in a single-source, single-sink flow network.

#### 24.2-7

Prove Lemma 24.2.

#### 24.2-8

Suppose that we redefine the residual network to disallow edges into  $s$ . Argue that the procedure FORD-FULKERSON still correctly computes a maximum flow.

#### 24.2-9

Suppose that both  $f$  and  $f'$  are flows in a flow network. Does the augmented flow  $f \uparrow f'$  satisfy the flow conservation property? Does it satisfy the capacity constraint?

#### 24.2-10

Show how to find a maximum flow in a flow network  $G = (V, E)$  by a sequence of at most  $|E|$  augmenting paths. (*Hint:* Determine the paths *after* finding the maximum flow.)

#### 24.2-11

The **edge connectivity** of an undirected graph is the minimum number  $k$  of edges that must be removed to disconnect the graph. For example, the edge connectivity of a tree is 1, and the edge connectivity of a cyclic

chain of vertices is 2. Show how to determine the edge connectivity of an undirected graph  $G = (V, E)$  by running a maximum-flow algorithm on at most  $|V|$  flow networks, each having  $O(V + E)$  vertices and  $O(E)$  edges.

#### **24.2-12**

You are given a flow network  $G$ , where  $G$  contains edges entering the source  $s$ . Let  $f$  be a flow in  $G$  with  $|f| \geq 0$  in which one of the edges  $(v, s)$  entering the source has  $f(v, s) = 1$ . Prove that there must exist another flow  $f'$  with  $f'(v, s) = 0$  such that  $|f| = |f'|$ . Give an  $O(E)$ -time algorithm to compute  $f'$ , given  $f$  and assuming that all edge capacities are integers.

#### **24.2-13**

Suppose that you wish to find, among all minimum cuts in a flow network  $G$  with integer capacities, one that contains the smallest number of edges. Show how to modify the capacities of  $G$  to create a new flow network  $G'$  in which any minimum cut in  $G'$  is a minimum cut with the smallest number of edges in  $G$ .

---

### **24.3 Maximum bipartite matching**

Some combinatorial problems can be cast as maximum-flow problems, such as the multiple-source, multiple-sink maximum-flow problem from Section 24.1. Other combinatorial problems seem on the surface to have little to do with flow networks, but they can in fact be reduced to maximum-flow problems. This section presents one such problem: finding a maximum matching in a bipartite graph. In order to solve this problem, we'll take advantage of an integrality property provided by the Ford-Fulkerson method. We'll also see how to use the Ford-Fulkerson method to solve the maximum-bipartite-matching problem on a graph  $G = (V, E)$  in  $O(VE)$  time. Section 25.1 will present an algorithm specifically designed to solve this problem.

#### **The maximum-bipartite-matching problem**